Out-Hacking a Ransomware Collective

Walkthrough of SANS x Flare x WiCyS "Sisterhood of the Traveling Packets" CTF

A step-by-step breakdown of the OPSEC failures I exploited to capture the flag

The Challenge

"Sisterhood of the Traveling Packets" is a virtual CTF powered by Flare and SANS Institute, in partnership with Women in CyberSecurity (WiCyS). It centers on investigating a fictional ransomware collective's dark web leak site through forensics, exploitation, and decoding to find a flag in the format flare{...}.

Here's exactly how I found mine.

Step 1: Getting In

The entry point was a .onion address, only reachable via Tor Browser:

```
http://b3u42lmdurcyqieiox3o3ns2c2rst6doyyjnwfuirr7bnewt53gke7yd.onion/
```

The site — branding itself "pantalones" , was a fully built-out ransomware leak page: a countdown-timer list of victim companies, some already "leaked," others "negotiating." Standard nav: Leaks, Crew, About.

Step 2: View-Source First, Always

Before clicking anything, I checked the raw HTML (Ctrl+U). Buried in there:

```
<!-- bm90X3RoZV9mbGFnX2tlZXBfbG9va2luZw== -->
```

Base64-decoded, this reads: not_the_flag_keep_looking. A deliberate decoy , confirms you're checking the right places without handing you anything.

The Crew page listed four operator handles and roles. These names became critical later:

- vex : operator / panel dev
- crypt : payload engineer
- mora : negotiations
- skid :initial access

Step 3: Forced Browsing : robots.txt

Instead of guessing hidden paths blindly, I checked the one file websites use to tell search engines what not to index:

```
http://b3u42lmdurcyqieiox3o3ns2c2rst6doyyjnwfuirr7bnewt53gke7yd.onion/robots.txt

User-agent: *
Disallow: /api.php
Disallow: /admin.php
```

Step 4: Reading the Stylesheet

I also pulled style.css directly. It contained CSS rules for things not yet visible anywhere on the public pages:

- A password input field styling
- An .activity-log table
- An .infra-table
- A .rainbow-flag class with an animated rainbow color-cycle

This told me, before I'd even logged into anything, that a real dashboard existed behind admin.php and that whatever text got styled with .rainbow-flag was almost certainly the flag itself.

Step 5: The Admin Panel

Navigating to /admin.php triggered a JavaScript alert() popup:

```
“WARNING: This area is for authorized users only. Only members of
pantalones may access this panel. No researchers or police allowed.”
```

This is purely cosmetic , a client-side popup provides zero actual access control. Clicking OK revealed a real server-side POST login form (username + password).

Step 6: Testing for SQL Injection

I tried classic SQLi payloads in the username field:

```
' OR '1'='1
admin'--
' OR 1=1#
```

Every attempt returned the exact same error, with my input echoed back verbatim: username '...' does not exist. This told me the backend checks the username against a fixed allow-list, not a raw SQL query , so classic injection wasn't the way in here. Good to rule out quickly rather than waste time on it.

Step 7: Username Enumeration

Since the error message differed between “user does not exist” and “incorrect password,” I could enumerate valid accounts using the four crew handles from Step 2:

Username	Result
vex	valid (incorrect password)
crypt	valid (incorrect password)
mora	valid (incorrect password)
skid	valid (incorrect password)
admin	invalid (does not exist)

Four confirmed real accounts, zero passwords yet.

Step 8: Enumerating the API

robots.txt had also flagged /api.php. Hitting it with no parameters:

```
{
  "error": "missing required parameter: action",
  "valid_actions": ["upload", "status", "messages", "decrypt",
                    "wallets", "payloads", "exfil"]
}
```

The API documented its own attack surface in its error response. I went through each action systematically. Most (status, wallets, payloads, exfil) returned operational data , Bitcoin wallets, staged payloads, exfil job history ,interesting for context, but no flag.

The messages action was the important one:

```
?action=messages&conversation_id=0
```

This returned actual internal crew chat logs ,completely unauthenticated. No login, no key, nothing. I incremented conversation_id (0, 1, 2, 3, 4...) and pulled every available thread.

Step 9: The Critical Leak : A Password in Plain Chat

Buried in conversation_id=2, operator mora asked crypt for a forgotten FTP password. crypt replied:

```
"UGFudGFsMG4zc19SdWwzeiE= - thats YOUR password mora.
 i encoded it this time, figure it out yourself."
```

mora responded: “ty. ive been using this password since 2011 and nobody has cracked it yet so i think im good lol.”

Decoding the Base64:

```
UGFudGFsMG4zc19SdWwzeiE=  →  Pantal0n3s_Rul3z!
```

Step 10: Logging In

Combining the confirmed valid username from Step 7 with the decoded password:

```
Username: mora
Password: Pantal0n3s_Rul3z!
```

This authenticated successfully into admin.php.

Step 11: Capturing the Flag

The dashboard displayed every victim organization with its ransom demand and a “decryption key” column. Every row showed a real-looking hex string , except one. The entry for “Sisterhood of the Traveling Packets” (the one victim entry that directly addressed the investigator on the public leak page) displayed, styled in the exact .rainbow-flag CSS class identified back in Step 4:

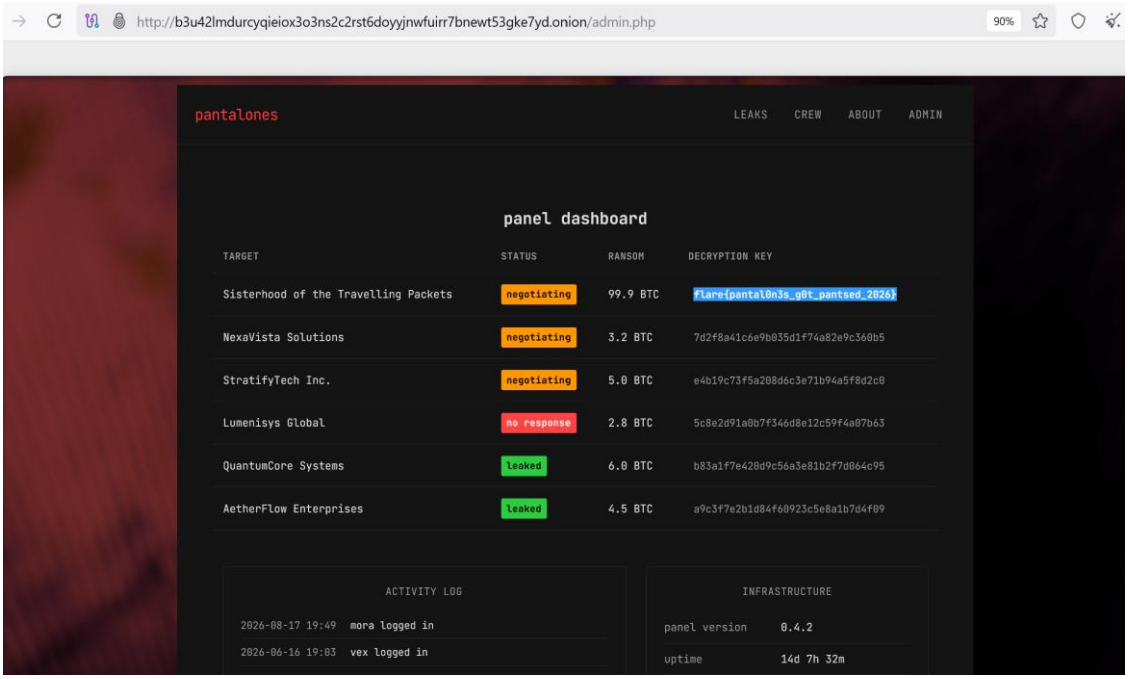


Figure 1 Authenticated admin.php dashboard showing the captured flag

flare{pantal0n3s_g0t_pantsed_2026}

Key Takeaways

None of this required breaking cryptography or exploiting infrastructure. Every step was patient, methodical forensics:

1. robots.txt and stylesheets leak more than people realize , hiding something from a crawler, or shipping CSS for a feature, both confirm that feature exists.
2. Client-side warnings are not security , a JS alert() stops nobody.
3. Verbose error messages are a gift to attackers , an API listing its own valid parameters removes all the guesswork.
4. Unauthenticated internal endpoints are a silent killer ,the messages action needed zero credentials to expose the whole crew's operational chatter.
5. Base64 is encoding, not encryption ,and reused passwords defeat even a decade of “nobody's cracked it yet.”

This mirrors how real ransomware operators actually get caught: not through some elegant cryptographic break, but through the same sloppy, human habits everyone else falls into.